

Adaptive Model Selection for Realized Volatility Forecasting via Deep Q-Learning

Julian Sester*, Zhensi Tan^{†,‡}, Huansang Xu*, Zhiyuan Zhang[†]

July 29, 2026

Abstract

We study realized volatility forecasting as an adaptive model selection problem motivated by the fact that different forecasting models can perform better in different volatility regimes. Rather than relying on a single forecasting specification, we develop a two-stage reinforcement-learning framework in which a heterogeneous pool of base forecasters is first estimated on rolling windows and then a Deep Q-Network (DQN) agent selects among them according to the prevailing volatility state. The candidate pool contains six models: OLS, HAR, LSTM, TCN-short, TCN-med, and TCN-long. The framework has two distinct roles. In one-step-ahead forecasting, the agent acts as a state-dependent selector whose actions do not affect future states. In recursive multi-step forecasting, the selected forecast is fed back into the next state, so current model choices affect the future forecasting path and the problem retains a genuinely sequential structure. The state representation combines recent log-realized volatility, lagged model errors, data-fidelity indicators, and contemporaneous candidate forecasts. Empirically, we use 5-minute price data for 40 Chinese stocks related to the SSE 50 Index over 2016–2024. The results show that the proposed framework delivers competitive forecasting performance relative to the strongest standalone models and an equally weighted ensemble, with particularly strong QLIKE performance in the reported one-step and recursive experiments.

All authors contributed equally to this work.

*Department of Mathematics, National University of Singapore.

[†]School of Statistics and Data Science, Shanghai University of Finance and Economics.

[‡]Corresponding author: Tan Zhensi. E-mail: tantan_zs@163.com.

1 Introduction

Realized volatility forecasting is central to risk management, derivatives pricing, portfolio allocation, and market surveillance(Andersen et al., 2003; Christoffersen et al., 2010). JS:Add References supporting this claimZS:Added. A key empirical difficulty is that volatility dynamics are highly state-dependent: markets move between calm periods, persistent high-volatility episodes, abrupt shocks, and post-shock normalization. As a result, no single forecasting model should be expected to perform uniformly well across all regimes. Linear models such as OLS and HAR are often robust, parsimonious, and effective when volatility is persistent and well described by recent averages. Neural architectures such as LSTM and TCN models can be more flexible in the presence of nonlinear dynamics or changing memory lengths, but their advantage may vary with the volatility environment and the amount of recent noise. The relevant forecasting question is therefore not only which model performs best on average, but which model should be trusted in the current regime.

ZS:To illustrate this regime dependence empirically, Figure 1 reports a heatmap of the mean QLIKE loss for the candidate forecasting models used in the empirical study. The volatility regimes are constructed according to tertiles of the average realized volatility over the previous 60 trading days. The annotated values are the raw mean QLIKE losses, while the cell colours are normalized within each volatility-regime column to highlight the relative ranking of models within that regime.

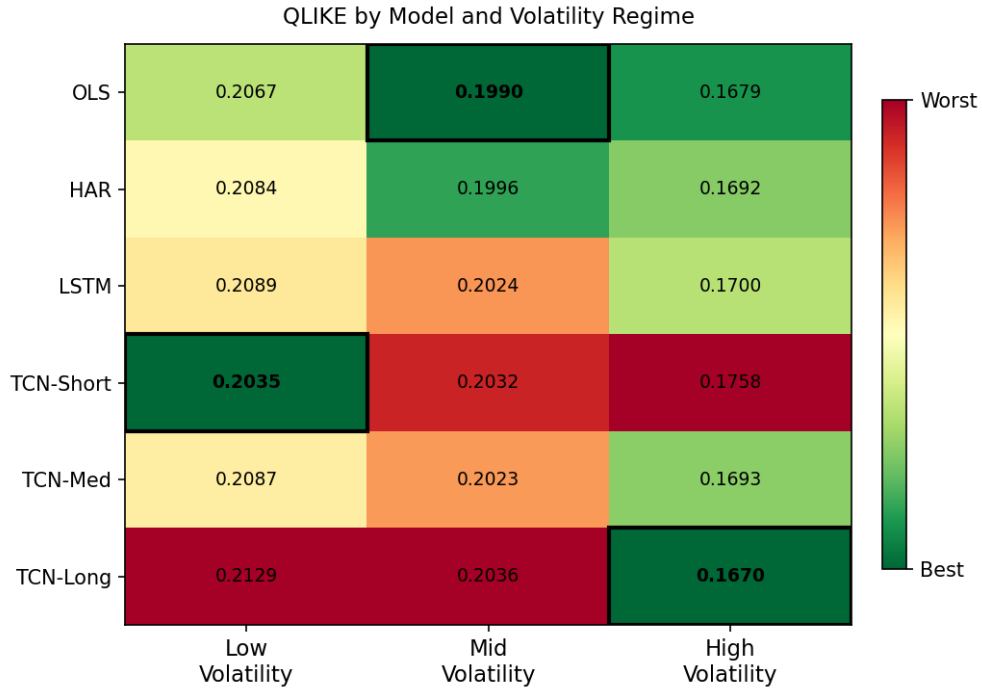


Figure 1: The figure reports the mean one-step-ahead QLIKE loss of each candidate forecasting model across low-, medium-, and high-volatility regimes. Volatility regimes are defined by tertiles of the average realized volatility over the previous 60 trading days. Lower QLIKE values indicate better predictive performance. The annotated values are mean QLIKE losses, while the cell colours are normalized within each volatility-regime column to emphasize relative model performance within that regime. The best-performing model in each regime is highlighted by a black box.

This regime dependence is consistent with the broader volatility forecasting literature. Volatility forecasting has traditionally relied on a wide range of competing econometric and statistical models, from GARCH-type specifications (Bollerslev, 1986a) to realized-volatility models such as HAR-RV (Corsi, 2009a), and empirical comparisons repeatedly show that predictive accuracy can be sensitive to the asset, forecasting horizon, sample period, loss function, and market regime (Poon and Granger, 2003; Hansen and Lunde, 2005; Christensen et al., 2023). This motivates an adaptive model-selection perspective in which the forecaster does not commit ex ante to a single volatility model, but instead learns which model is most suitable under the current information set. Related ideas appear in automated algorithm selection (Kerschke et al., 2019) and dynamic model averaging (Raftery et al., 2010; Koop and Korobilis, 2012), where model choices or model weights evolve over time. A fixed benchmark chooses one model for

all periods, while a static forecast combination assigns the same aggregation rule regardless of whether the market is calm, turbulent, or transitioning. Such approaches can dilute the strengths of specialized models precisely when those strengths are most useful.

Reinforcement learning provides a natural sequential formulation of this adaptive selection problem. The action space can be defined as a finite set of candidate volatility models, $a_t \in \{M_1, \dots, M_K\}$; the state can summarize recent market conditions, rolling forecast performance, and recursive-feedback information; and the reward can be based on negative out-of-sample forecasting loss, such as MSE or QLIKE (Patton, 2011). This model-as-action formulation is closely related to RL-based dynamic model selection in forecasting, where Q-learning or deep Q-learning agents select a model from a forecasting pool (Feng et al., 2020; Chien et al., 2020; Pannakkong et al., 2023), as well as to RL-based model-combination methods that learn time-varying ensemble weights for non-stationary time series (Fu et al., 2022). In one-step-ahead forecasting, the agent acts as a state-dependent selector. In recursive multi-step forecasting, the selected forecast is fed back into future state inputs, so a current model choice can affect not only immediate accuracy but also the future forecasting path. Our goal is therefore not to replace existing volatility models, but to translate their regime-specific strengths into a dynamic model selection policy.

1.1 Contributions

This paper develops a reinforcement-learning framework for realized volatility forecasting. Rather than relying on a single forecasting model, we treat forecasting as a sequential model selection problem: at each date, a Deep Q-Network (DQN) agent selects one model from a heterogeneous pool of candidate predictors according to the prevailing volatility state.

The main contributions are threefold. First, we propose a two-stage forecasting architecture. In the first stage, six candidate models—OLS, HAR, LSTM, TCN-short, TCN-med, and TCN-long, all of them being described in Section 3.1—are trained on rolling historical windows. In the second stage, these pre-trained models form the discrete action space of a DQN agent, which learns a state-dependent model selection rule from recent log-realized volatility observations.

Second, we formulate both one-step-ahead and recursive multi-step forecasting within a unified reinforcement-learning framework. For one-step-ahead forecasting, the agent acts as an adaptive selector whose actions do not affect future states. For recursive forecasting, the selected model’s prediction is fed back into the next state, so current decisions influence the future forecasting path. This allows the agent to learn policies that account not only for immediate forecast accuracy, but also for the accumulation of errors over longer horizons.

Third, we provide an empirical study using high-frequency 5-minute price data for 40 Chinese stocks associated with the SSE 50 Index from 2016 to 2024. We evaluate the proposed approach against individual statistical and neural-network benchmarks, as well as an equally weighted ensemble, using standard volatility forecasting losses including MSE and QLIKE. The empirical design allows us to assess whether dynamic reinforcement-learning-based model selection improves upon both fixed individual models and static forecast combinations.

1.2 Relevant Literature

This paper is related to four strands of literature: realized volatility forecasting, machine learning for financial time series, forecast combination and model selection, and reinforcement learning for adaptive decision making.

The first strand concerns the measurement and forecasting of realized volatility. Realized volatility uses high-frequency intraday returns to approximate latent integrated volatility and has become a central object in empirical volatility research. Andersen et al. (2001) show that realized volatility constructed from intraday data provides an informative measure of daily volatility. Traditional volatility models, such as the GARCH model of Bollerslev (1986b), model conditional variance using daily returns, while later approaches incorporate realized measures directly into volatility models. For example, Hansen et al. (2012) propose the Realized GARCH framework, which links latent volatility and observed realized measures through a measurement equation. A particularly important benchmark for realized volatility forecasting is the heterogeneous autoregressive model of Corsi (2009b), which captures volatility persistence through daily, weekly, and monthly realized volatility components.

The second strand studies machine learning methods for volatility forecasting. Neu-

ral networks are attractive because volatility dynamics may be nonlinear, persistent, and regime-dependent. Bucci (2020) studies realized volatility forecasting with neural networks and shows that recurrent architectures can be competitive with traditional econometric models. More recently, Zhang et al. (2024) examine volatility forecasting using machine learning and intraday commonality, highlighting the value of cross-sectional information across assets. Our empirical design is related to this idea, since the DQN selector is trained on pooled observations from multiple stocks in order to learn a more general model selection policy.

The third strand concerns forecast combination and adaptive model selection. Volatility forecasting has traditionally relied on a wide range of competing econometric, statistical, and machine-learning models, and empirical comparisons show that predictive accuracy can be sensitive to the asset, forecasting horizon, sample period, loss function, and market regime. This motivates an adaptive model-selection perspective in which the forecaster does not commit *ex ante* to a single volatility model, but instead learns which model is most suitable under the current information set. Forecast combination addresses a related concern by pooling information across models, but simple averaging imposes a fixed aggregation rule and cannot respond to changes in the volatility state. More flexible approaches allow model weights or model choices to evolve over time, as in dynamic model averaging and automated algorithm selection. Our approach can be viewed as a state-dependent alternative to static forecast combination: the candidate volatility models form a finite set of forecasting actions, the state summarizes recent market conditions and rolling forecast performance, and the reward is based on negative out-of-sample forecasting loss, such as MSE or QLIKE (Patton, 2011). Instead of assigning fixed weights to all models, the agent selects the candidate model expected to perform best under the current information set.

The fourth strand concerns reinforcement learning for sequential decision making. Reinforcement learning provides a framework for learning policies that maximize cumulative rewards (Sutton and Barto, 2018), and the DQN algorithm of Mnih et al. (2015) extends Q-learning with neural-network function approximation. In our setting, the action is not a trading decision but the selection of a forecasting model. For one-step-ahead forecasting, this formulation is close to a contextual model-selection problem, since the action does not affect future states. The sequential reinforcement-learning interpretation becomes more central in recursive multi-step forecasting, where

the selected forecast enters the next state and current actions influence future inputs and rewards. Our contribution is to bring this adaptive selection perspective to realized volatility forecasting with a heterogeneous pool of econometric and neural-network forecasters.

1.3 Structure

The remainder of the paper is organized as follows. In Section 2, we introduce the key concepts and notations, including the definition of realized volatility and the Markov Decision Process (MDP) formulation for model selection. In Section 3, we present the proposed *reinforcement learning framework for volatility forecasting* (RLVF), detailing the candidate forecasting models and the DQN architecture for model selection. In Section 4, we describe the empirical design and report the forecasting results, including the data sources, evaluation metrics, benchmark models, and performance comparisons.

2 Realized Volatility Forecasting Problem

In this section, we introduce the key concepts and notations that underpin our reinforcement learning framework for volatility forecasting. We first define the realized volatility measure and its properties, which serve as the target variable for our forecasting task. We then present the Markov Decision Process (MDP) formulation that allows us to model the adaptive selection of forecasting models as a sequential decision-making problem.

2.1 Realized Volatility

Let $X_t = \log S_t$ denote the log-price of some asset S_t at time t . For trading day i , suppose that intraday log-prices are observed at times

$$i - 1 = t_{i,0} < t_{i,1} < \cdots < t_{i,n_i} = i.$$

The realized variance for day i is then defined as

$$\text{RV}_i = \sum_{j=1}^{n_i} \left(X_{t_{i,j}} - X_{t_{i,j-1}} \right)^2.$$

Following the realized-volatility literature (e.g., Andersen et al., 2001), we use RV_i as an observable proxy for the latent daily price variation. Since realized volatility is strictly positive and strongly right-skewed, with a long right tail and occasional extremely large observations, JS:Why does this make necessary to apply the log-transform?ZS:I deleted “persistence”. The main motivation is the pronounced right skewness of realized volatility. we follow the common practice (Andersen et al., 2001; Bucci, 2020; Zhang et al., 2024) of modeling its logarithm to mitigate the right skewness and reduce the disproportionate influence of extremely large observations:

$$V_i = \log(RV_i),$$

which is the forecasting target throughout the paper.

2.2 Deep Reinforcement Learning Framework

The primary objective of this study is to provide accurate out-of-sample forecasts for future realized volatility. While individual forecasting models capture distinct facets of volatility dynamics, their predictive performance often exhibits time-varying characteristics. To leverage the complementary strengths of multiple candidates, we formulate the task of adaptive model selection as a Markov Decision Process (MDP).

The MDP formulation provides a principled and flexible structure for sequential decision-making under uncertainty, enabling dynamic adjustment of the model selection strategy in response to evolving market conditions. Under this framework, an agent interacts with the historical financial environment to learn an optimal policy. At each time step i , the agent evaluates the current market state s_i and selects the most promising model from a candidate pool. The precise goal is to minimize the cumulative forecasting error of the predicted values $(\hat{V}_{i+j})_{j=1,\dots,h}$ relative to the realized values $(V_{i+j})_{j=1,\dots,h}$, where h denotes the forecasting horizon. In particular, the MDP framework allows for the incorporation of state-dependent information, supports the design of reward functions that balance short-term predictive accuracy with long-term performance, and naturally accommodates exploration–exploitation trade-offs, thereby enhancing the robustness, adaptability, and overall effectiveness of the forecasting process.

2.2.1 The MDP Formulation

To formalize the sequential decision-making process, we define the MDP as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- **State Space ($\mathcal{S}$):** At time t , the state $s_t \in \mathcal{S}$ contains the quantities deemed to be relevant for forecasting volatility, namely the market environment and the forecasting history. In particular, s_t may include recent realized volatility, model-specific features (e.g., past prediction errors), and other observable market indicators that are informative for model selection. JS:Refer here to the part in Section 3.2 of the paper where the state is described in more detail.
- **Action Space ($\mathcal{A}$):** The action $a_t \in \mathcal{A}$ represents the selection of a specific model from the candidate pool $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$. Accordingly, the action space $\mathcal{A} = \{1, 2, \dots, K\}$ is discrete and time-independent as the pool does not change over time.
- **State Transition Probability ($\mathcal{P}$):** $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ defines the probability of transitioning to state s_{t+1} given the current state s_t and the chosen action a_t . JS:We should discuss that we do not assume a specific form for $\mathcal{P}$, but rather learn the optimal policy directly from observed transitions via the used RL algorithm.
- **Reward Function ($\mathcal{R}$):** The (random) reward $r_t := r(s_t, a_t, s_{t+1})$ is designed to measure the quality of the selected forecasting model relative to the candidate pool. Let

$$e_t^{(j)} := \ell_{\text{RL}}(V_{t+1}, \hat{V}_{t+1}^{(j)}), \quad j = 1, \dots, K, \quad (1)$$

denote the forecasting error of candidate model j at time t , where ℓ_{RL} is the loss used to train the selector. For example, ℓ_{RL} may be the squared error or the QLIKE loss, depending on the forecasting criterion emphasized in the empirical implementation. Define

$$\tilde{e}_t := \min_{j=1, \dots, K} e_t^{(j)}.$$

The reward assigned to action a_t is

$$r_t = \tilde{e}_t - e_t^{(a_t)} = -\left(e_t^{(a_t)} - \tilde{e}_t\right). \quad (2)$$

JS:When we are switching to the recursive setting $h' > 0$, then $\tilde{e}_t$ also becomes action dependent? So We are not exactly minimizing QLIKE anymore. I am not sure whether this is an issue. One could avoid this bu switching to $-e_t^{(at)}$, but not sure whether this is better. Thus $r_t \leq 0$, and the ex-post best candidate model receives reward zero. All other choices receive a negative reward equal to their excess loss relative to the best candidate available at that time. Hence the reward is the negative of an instantaneous model-selection regret. This normalization removes part of the variation due to state difficulty and focuses the agent on choosing the best model within the current volatility environment.

- **Discount Factor (γ):** The discount factor $\gamma \in [0, 1]$ controls the trade-off between short-term forecasting accuracy and long-term performance stability.

The goal of the agent is to learn an optimal policy. In a finite-horizon decision problem, a general policy is a sequence of decision rules

$$\Pi := \{\pi = (\pi_0, \dots, \pi_{h-1}) : \pi_t : \mathcal{H}_t \rightarrow \Delta(\mathcal{A}), t = 0, \dots, h-1\},$$

where $\mathcal{H}_t$ denotes the history available at decision time t and $\Delta(\mathcal{A})$ denotes the probability simplex over the discrete action space. Thus policies are sequential objects in general. Since our state s_t is constructed to summarize the relevant volatility history, model-performance information, and recursive-feedback information, we work with the Markov subclass

$$\Pi_M := \{\pi = (\pi_0, \dots, \pi_{h-1}) : \pi_t : \mathcal{S} \rightarrow \Delta(\mathcal{A}), \pi_t(\cdot | s) \text{ is measurable in } s \in \mathcal{S}, t = 0, \dots, h-1\}.$$

The DQN implementation further uses a parameterized stationary Markov policy, interpreted on the augmented state that includes the elapsed recursive step. A deterministic model-selection rule is the special case in which $\pi_t(\cdot | s)$ puts unit mass on one action, equivalently $\pi_t^{\text{det}} : \mathcal{S} \rightarrow \mathcal{A}$. Formally, the value of a state $s \in \mathcal{S}$ under a Markov policy $\pi \in \Pi_M$ is defined as:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{h-1} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s, a_t \sim \pi_t(\cdot | s_t) \right],$$

where h is the horizon length (e.g., the number of forecasting steps). The objective of our RL framework is to find a parameterized policy $\pi_\phi \in \Pi_M$ that maximizes the

expected value over the initial state distribution μ , i.e., approximately an optimizer of

$$\sup_{\pi \in \Pi_M} J(\pi), \text{ where } J(\pi) := \mathbb{E}_{s \sim \mu}[V^\pi(s)]$$

In our implementation, μ represents the empirical distribution of historical market states constructed from rolling windows in the training (and evaluation) dataset. This formulation explicitly accounts for the sequential and path-dependent nature of model selection, ensuring that the learned policy prioritizes sustained forecasting performance over the horizon rather than myopic, one-step-ahead accuracy.

2.2.2 State Dynamics and Recursive Feedback

The state transition mechanism $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ in our framework is intrinsically linked to the chosen forecasting scheme. Depending on the forecasting horizon and implementation strategy, the interaction between the agent’s actions and the environment can be categorized into two settings:

- **Direct Forecasting:** This category includes single-step forecasting ($h = 1$) and multi-step direct forecasting, where the agent produces the sequence $(\hat{V}_{t+i})_{i=1, \dots, h}$ in a single decision step based solely on observed historical information. In this case, the environment is effectively passive, as there is no feedback from the generated forecasts into future states. Consequently, the state evolution is exogenous to the agent’s actions, and the problem reduces to a contextual bandit or, equivalently, a supervised model selection task.
- **Multi-step Recursive Forecasting:** For longer horizons, we alternatively implement a recursive forecasting mechanism. In this scheme, the agent’s action a_t (i.e. selecting a candidate model) generates a one-step-ahead forecast $\hat{V}_{t+1}$, which is incorporated into the next state s_{t+1} . This induces a feedback loop whereby forecasts influence subsequent inputs. As a result, the state transition becomes partially endogenous, and the problem retains the full sequential structure of an MDP, with current decisions directly affecting future states and rewards.

The distinct nature of these two schemes dictates the specific role and optimization objective of the RL agent. In the direct forecasting scheme, the agent functions as

an adaptive model selector that evaluates the relative strengths of candidate models across diverse market states. Since each step is decoupled from future states, the task reduces to a series of independent decisions aimed at minimizing instantaneous error, effectively operating as a contextual bandit. But this approach often requires re-training or maintaining specific models for different horizons and provides limited insights into the sequential dynamics of volatility.

In contrast, the recursive forecasting scheme offers greater flexibility across varying horizons but introduces the well-known issue of error propagation. In this setting, the advantages of the RL framework become more pronounced. By explicitly modeling the recursive structure as an MDP, the agent accounts for the intertemporal dependence induced by its own predictions and is trained to optimize cumulative reward rather than one-step accuracy. This forward-looking objective encourages the selection of models that promote stability over the entire forecasting path, thereby mitigating the accumulation of errors that commonly arises in classical recursive forecasting approaches.

3 Methodology

JS:All models should be described in some more detail or at least referenced. Maybe we can add a table with the model names, their class, and a short description. Also a more detailed description in the appendix would be good. We propose a *reinforcement learning framework for volatility forecasting (RLVF)* based on a two-stage architecture. In the first stage, a heterogeneous pool of candidate models is pre-trained to capture diverse aspects of market dynamics. This model pool defines the discrete action space for a Deep Q-Network (DQN) agent in the second stage, which learns to select the most appropriate model based on high-dimensional market observations.

By formulating the model selection process as a Markov Decision Process (MDP), the proposed framework enables dynamic, state-dependent forecasting that adapts to evolving market conditions. This design allows the agent to exploit the complementary strengths of individual models while accounting for temporal dependencies in forecasting performance, thereby improving robustness and adaptability across different market regimes.

3.1 Candidate Forecasting Models

To provide the RL agent with a diverse set of candidate strategies, we construct a model pool $\mathcal{M}$ consisting of six candidate forecasting models spanning four heterogeneous model classes. These models range from classical statistical approaches to advanced deep learning architectures, thereby capturing complementary features of volatility dynamics across different market conditions.

3.1.1 Linear Statistical Benchmarks

We include Ordinary Least Squares (OLS) as a fundamental baseline. More importantly, we incorporate the Heterogeneous Autoregressive (HAR) model (Corsi, 2009b), which is widely recognized for its ability to capture the long-memory property of realized volatility through daily, weekly, and monthly averages.

3.1.2 Deep Neural Architectures

To capture non-linear dynamics and complex temporal patterns, we employ two neural network families:

- Long Short-Term Memory (LSTM): A recurrent architecture specifically suited for capturing long-term temporal dependencies in time series data.
- Temporal Convolutional Network (TCN): A convolution-based model that utilizes dilated causal convolutions to achieve a large receptive field while preventing information leakage from the future. In the empirical study, we implement this architecture in three variants, namely TCN-short, TCN-med, and TCN-long, to capture different effective memory lengths.

3.2 Model Selection via DQN

We formalize the adaptive selection of candidate models as an MDP. [Specifically, we introduce a state space formulation carefully designed to accommodate both direct and recursive forecasting paradigms as visually summarized in Figure 2.](#) At each time $t = t_0 + h'$, where t_0 denotes the start of a forecasting episode and $h' \in \{0, 1, \dots, h-1\}$

represents the number of elapsed steps within a horizon h , the agent observes a comprehensive state vector $s_t \in \mathbb{R}^{D \times (K+2) + K}$. **JS:where D denotes the look-back horizon and K the number of models considered.** This state encapsulates historical market dynamics, model performance indicators, information fidelity signals, and prospective candidate forecasts, structured into two primary branches: **JS:Make explicit that the state is a matrix of size $D \times (K+2)$ plus a vector of size K , so the total dimension is $D \times (K+2) + K$, and that it consists of a historical branch and a look-ahead branch which are described below.**

- **Historical Branch:** The agent maintains a structured feature matrix $\mathbf{X}_t = [\mathbf{V}_t, \mathbf{E}_t, \mathbf{C}_t] \in \mathbb{R}^{D \times (K+2)}$ that captures the market dynamics over a look-back window of length D . This matrix is partitioned into three functional components:

- (i) **Historical Volatility Trajectory** ($\mathbf{V}_t \in \mathbb{R}^{D \times 1}$): The sequence of log-realized volatilities. In the recursive stage, this vector evolves from purely observed data to a mixture of observations and agent-generated forecasts.
- (ii) **Historical Error Matrix** ($\mathbf{E}_t \in \mathbb{R}^{D \times K}$): A systematic collection of past prediction errors for the candidate pool. Let $e_{i,j}$ denote the element in the i -th row and j -th column of $\mathbf{E}_t$ (where $i \in \{1, \dots, D\}$ and $j \in \{1, \dots, K\}$). Each row i corresponds to a temporal lag $k = i - 1$, such that:

$$e_{i,j} = \hat{V}_{t-(i-1)}^{(j)} - \tilde{V}_{t-(i-1)} \quad (3)$$

where $\tilde{V}_{t-(i-1)}$ represents the target volatility (realized or proxied) at time $t - (i - 1)$. This mapping ensures that the i -th row of $\mathbf{E}_t$ is temporally aligned with the i -th elements of $\mathbf{V}_t$ and $\mathbf{C}_t$.

- (iii) **Information Fidelity Indicator** ($\mathbf{C}_t \in \mathbb{R}^{D \times 1}$): A reliability vector that maps each temporal observation to a confidence score. The vector is constructed as $\mathbf{C}_t = [c_t, c_{t-1}, \dots, c_{t-D+1}]^\top$, where the element corresponding to lag $k \in \{0, \dots, D - 1\}$ is defined as:

$$c_{t-k} = 1 - \frac{\max\{h' - k, 0\}}{h}.$$

Here, c_{t-k} quantifies the reliability of the data point at time $t - k$. A value of 1 signifies a ground-truth observation, while $c_{t-k} < 1$ indicates a synthetic

value generated during the recursive process, with the fidelity decreasing as the recursive depth h' increases relative to the lag k .

- **Look-ahead Branch:** A vector $\hat{\mathbf{V}}_{t+1} \in \mathbb{R}^K$ containing the contemporaneous forecasts generated by all candidate models for the subsequent period.

Depending on the forecasting horizon and implementation strategy, the information streams within these branches are updated using either observed market data or synthetic feedback generated by the agent’s previous actions. **JS:Don’t use bullet points here, but rather describe the two cases in a paragraph.**

- **Direct/One-step Forecasting:** In the one-step configuration ($h' = 0$), the state s_t is strictly exogenous **JS:i.e. the agent does not take any influence on the evolution of the state**. The historical branch $\mathbf{X}_t$ is constructed entirely from realized observations, where $\mathbf{V}_t = (V_{t-k})_{k=0,\dots,D-1}$ represents the vector of historical log-RV and $\mathbf{E}_t$ contains the true lagged prediction errors relative to ground-truth values. Since $h' = 0$, the fidelity indicator c_{t-k} in $\mathbf{C}_t$ remains uniformly at 1 for all $k \in \{0, \dots, D-1\}$, reflecting a state of maximum data reliability. In this setting, the transition from s_t to s_{t+1} is driven solely by market evolution and is independent of the agent’s previous actions.
- **Recursive Forecasting ($h' > 0$):** As the forecasting horizon extends beyond the initial step, the state s_t becomes endogenous **JS:i.e. action-dependent**, as it must be constructed without the benefit of contemporaneous ground-truth observations. The components of s_t are dynamically adapted as follows:
 1. *Historical Trajectory Substitution:* The volatility vector $\mathbf{V}_t$ is updated using a “roll-and-fill” mechanism. The oldest observation is discarded, and the agent’s chosen forecast from the previous step, $\hat{V}_t^{(a_{t-1})}$, is appended to the tail of the window. This creates a partially synthetic historical sequence $\mathbf{V}_t = (\hat{V}_t, \dots, \hat{V}_{t_0+1}, V_{t_0}, \dots, V_{t_0-(D-h'-1)})$. **JS:Shouldn’t this be $\mathbf{V}_t = (\hat{V}_{t-1}^{(a_{t-1})}, \hat{V}_{t-2}^{(a_{t-2})}, \dots, \hat{V}_{t-h'}^{(a_{t-h'})}, V_{t-h'-1}, \dots, V_{t-D+1})$?**
 2. *Proxied Error Computation:* Since true realized volatility is unobservable during the recursive loop, the lagged error matrix $\mathbf{E}_t$ is constructed using proxied errors. These are defined as the discrepancy between each candidate

model’s forecast and the agent’s previously selected “best” forecast: $e_{i,j} = \hat{V}_{t-(i-1)}^{(j)} - \hat{V}_{t-(i-1)}^{(a_{t-i})}$.

3. *Data Fidelity Attenuation*: Crucially, the point-wise fidelity indicator c_{t-k} begins to decay according to the elapsed recursive depth h' , informing the agent of the increasing proportion of estimation-based entries within the current feature matrix $\mathbf{X}_t$.

This recursive feedback loop induces a direct dependence of future states on the agent’s past actions, thereby fully operationalizing the sequential decision-making structure of the MDP.

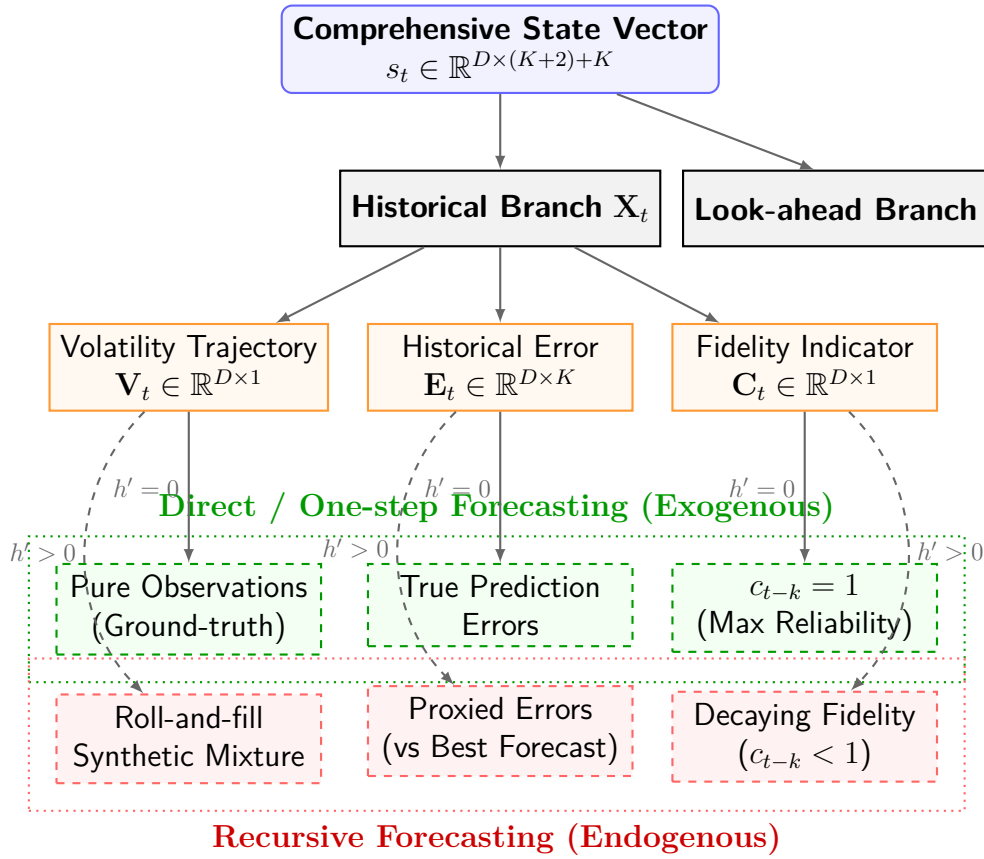


Figure 2: Adaptive MDP State Space Design (s_t). The framework systematically decouples the feature matrix into historical trajectory, error mappings, and a fidelity mechanism, adapting dynamically from purely exogenous inputs ($h' = 0$) to an endogenous feedback loop in deep recursive horizons ($h' > 0$).

JS:Should we move the figure to the beginning of the section? The optimization objective is to learn an optimal policy $\pi^* \in \Pi_M$ that maximizes the expected cumulative discounted reward. To this end, we employ a Deep Q-Network (DQN) to approximate the action-value function $Q(s, a; \theta)$ JS:where θ describes the parameters of a deep neural network. Unlike traditional tabular Q-learning, the DQN leverages a deep neural network to accommodate the continuous and high-dimensional state space associated with financial volatility. The Q-network evaluates state–action values to guide model selection, and the input layer in our study receives the $(D \times (K + 2) + K)$ -dimensional state vector s_t , which aggregates historical log-realized volatilities, lagged model errors, fidelity indicators, and contemporaneous candidate forecasts.

We approximate the DQN’s action-value function using a dual-stream architecture to accommodate the heterogeneous state s_t . Specifically, the historical branch $\mathbf{X}_t \in \mathbb{R}^{D \times (K+2)}$ is processed via a TCN. By leveraging hierarchical dilated causal convolutions, the TCN extracts deep temporal dependencies within the look-back window while preserving causality. This design is uniquely suited to capturing multi-scale volatility dynamics and managing the fidelity gradient encoded in $\mathbf{C}_t$. Simultaneously, an MLP processes the look-ahead branch $\hat{\mathbf{V}}_{t+1} \in \mathbb{R}^K$ to capture prospective consensus among candidates. These consensus features are concatenated with the TCN’s temporal representations and fed through an output layer to estimate action-values $Q(s_t, a; \theta)$. This fusion allows the policy to balance historical regularities with immediate forecasts, ensuring robust model selection across the recursive horizon.

During the inference phase, the agent adopts a deterministic greedy policy, selecting the model with the highest expected reward:

$$a_t^* = \arg \max_{a \in \mathcal{A}} Q(s_t, a; \theta).$$

By leveraging this deep architecture, our selection process facilitates a sophisticated, state-dependent evaluation of each candidate model’s potential performance under the prevailing market environment.

3.3 Why the reward induces adaptive model selection

The preceding MDP formulation casts volatility forecasting as a model-selection problem. The action a_t does not produce a new forecasting model; it selects one element

from a fixed candidate pool of K models. The role of reinforcement learning is therefore to learn when each candidate model should be used. The reward in (2) makes this precise: it is the negative excess loss of the selected model relative to the best candidate available at the same date.

To see this, first consider the one-step-ahead setting. In this case the state process is exogenous to the agent's action, so the problem reduces to a contextual model-selection problem. For a state $s \in \mathcal{S}$, define the conditional expected error of candidate model j by

$$m_j(s) := \mathbb{E}[e_t^{(j)} \mid s_t = s], \quad j = 1, \dots, K,$$

and define

$$b(s) := \mathbb{E}[\tilde{e}_t \mid s_t = s].$$

If the agent chooses action j in state s , then its conditional expected reward is

$$\mathbb{E}[r_t \mid s_t = s, a_t = j] = b(s) - m_j(s).$$

Since $b(s)$ does not depend on the chosen action, maximizing conditional expected reward is equivalent to minimizing conditional expected forecasting error:

$$\operatorname{argmax}_{j=1, \dots, K} \mathbb{E}[r_t \mid s_t = s, a_t = j] = \operatorname{argmin}_{j=1, \dots, K} m_j(s).$$

Thus the reward in (2) induces a state-dependent selector that chooses the model with the smallest conditional loss in the current volatility state.

This explains the mathematical source of potential outperformance. A fixed model chooses the same j in every state, while an adaptive selector chooses a measurable deterministic policy $\pi \in \Pi_{\text{det}}$, where $\Pi_{\text{det}} \subset \Pi_{\text{M}}$ denotes the stationary deterministic model-selection rules. The expected forecasting loss of such a policy is

$$R(\pi) := \mathbb{E}[m_{\pi(s_t)}(s_t)].$$

The oracle state-dependent selector is

$$\pi^*(s) \in \operatorname{argmin}_{j=1, \dots, K} m_j(s),$$

and therefore

$$R(\pi^*) = \mathbb{E} \left[\min_{j=1,\dots,K} m_j(s_t) \right].$$

Since

$$\mathbb{E} \left[\min_{j=1,\dots,K} m_j(s_t) \right] \leq \min_{j=1,\dots,K} \mathbb{E}[m_j(s_t)],$$

the oracle adaptive selector weakly dominates the best fixed model in the candidate pool. The inequality is strict whenever the identity of the best candidate model changes across states with positive probability. Hence the benefit of the proposed framework comes from exploiting model complementarity across volatility regimes, rather than from replacing the base forecasting models.

Equivalently, define the state-dependent selection gap

$$\Delta_{\text{sel}} := \min_{j=1,\dots,K} \mathbb{E}[m_j(s_t)] - \mathbb{E} \left[\min_{j=1,\dots,K} m_j(s_t) \right] \geq 0.$$

A positive value of Δ_{sel} means that no single candidate model is uniformly best and that there is statistical value in learning a state-dependent selection rule.

Proposition 1 (Reward optimality and oracle selection). *In the one-step-ahead setting, assume that the conditional risks $m_j(s)$ are well defined. Then maximizing the conditional expected reward in (2) is equivalent to choosing a model with minimal conditional expected forecasting error. Consequently, the oracle state-dependent selector satisfies*

$$R(\pi^*) \leq \min_{j=1,\dots,K} R(j), \quad R(j) := \mathbb{E}[m_j(s_t)].$$

If the learned selector $\hat{\pi}$ satisfies

$$\sup_s \left| m_{\hat{\pi}(s)}(s) - m_{\pi^*(s)}(s) \right| \leq \varepsilon,$$

then

$$R(\hat{\pi}) \leq R(\pi^*) + \varepsilon.$$

In particular, $\hat{\pi}$ outperforms the best fixed candidate model whenever

$$\Delta_{\text{sel}} > \varepsilon.$$

Proof. The first claim follows from

$$\mathbb{E}[r_t \mid s_t = s, a_t = j] = b(s) - m_j(s),$$

where $b(s)$ is independent of j . Thus maximizing expected reward is the same as minimizing $m_j(s)$. The oracle inequality follows from

$$\mathbb{E}\left[\min_j m_j(s_t)\right] \leq \min_j \mathbb{E}[m_j(s_t)].$$

The final bound follows directly from the assumed uniform approximation error:

$$R(\hat{\pi}) = \mathbb{E}[m_{\hat{\pi}(s_t)}(s_t)] \leq \mathbb{E}[m_{\pi^*(s_t)}(s_t)] + \varepsilon = R(\pi^*) + \varepsilon.$$

□

The recursive multi-step setting is different. There, the selected forecast is fed back into the next state, so future states depend on past actions. The problem is no longer a sequence of independent one-step model-selection decisions. Instead, the DQN estimates action values of the form

$$Q^\pi(s, a) = \mathbb{E}^\pi \left[\sum_{r=0}^{h-1} \gamma^r \left(\tilde{e}_{t+r} - e_{t+r}^{(a_{t+r})} \right) \mid s_t = s, a_t = a \right].$$

Thus the agent is trained to minimize cumulative discounted selection regret over the recursive forecasting path. This connects the reward in (2) with the recursive state construction: an action is valuable not only when it gives a low immediate error, but also when the generated forecast produces future states from which subsequent model choices remain accurate.

This is the main reason why the reinforcement-learning formulation is more natural in the recursive case than in the one-step case. For one-step forecasting, the method is essentially a state-dependent selector. For recursive forecasting, the action affects the future feature vector through the roll-and-fill mechanism, the proxied error matrix, and the fidelity indicator. The Bellman recursion used by the DQN therefore learns a policy that balances immediate relative forecast accuracy against the future consequences of feeding synthetic forecasts back into the state.

Finally, the interpretation of (2) is deliberately relative. The reward does not claim that the selected model has small absolute error in every state; rather, it rewards the

agent for selecting a model whose error is close to the best error available within the candidate pool. This is consistent with the paper’s objective: the proposed method is an adaptive selection layer over existing volatility forecasters, not a replacement for those forecasters.

4 Empirical Study

In this section, we present the empirical design and results of our reinforcement learning framework for volatility forecasting. We first describe the dataset and pre-processing steps, followed by the two-stage training scheme for the candidate forecasting models and the DQN agent. Finally, we evaluate the performance of our approach against various benchmarks using standard volatility forecasting metrics.

4.1 Data and Pre-processing

We utilize a high-frequency dataset of 5-minute trade prices for 40 Chinese stocks, obtained from Tick Data. These stocks are either historical or current constituents of the SSE 50 Index, representing the most liquid and large-cap segment of the Chinese equity market. The sample period spans from January 1, 2016, to December 31, 2024.

To ensure the integrity of the realized volatility measures, we restrict our analysis to active trading hours (09:30–11:30 and 13:00–15:00), effectively excluding overnight returns and the lunch break. We apply a rigorous filtering procedure to the raw data, removing trading days with excessive missing values or insufficient liquidity (e.g., cases where prices remain constant for over two consecutive hours). After these cleaning steps, we obtain an average of 2,126 trading days for each of the 40 stocks in our final sample.

4.2 Two-Stage Training Scheme: Base Models and RL Agent

The implementation of our framework follows a two-stage procedure: first, establishing a diverse pool of pre-trained forecasting models, and second, optimizing the RL-based adaptive selector by interacting with these fixed models.

Unlike static ensemble methods, our approach does not simply aggregate pre-computed out-of-sample forecasts. Instead, the first stage is dedicated to estimating

the parameters of each candidate model so that they effectively capture underlying market dynamics. In the second stage, these models define the discrete action space of the RL environment. This decoupled design is particularly important for recursive forecasting ($h > 1$), where the next state s_{t+1} depends on the agent’s previous action through the generated forecast. By freezing the parameters of the base models prior to RL training, we ensure that the agent focuses exclusively on learning an optimal model selection policy, without altering the underlying forecasting mechanisms.

4.2.1 Pre-Training Candidate Forecasting Models

Before optimizing the reinforcement learning framework, we establish a suite of candidate forecasting models as detailed in Section 3.1. While OLS, HAR, and LSTM are configured with a baseline look-back window of 22 lags, we further extend the TCN into three variants to capture multi-scale temporal dependencies. These are denoted as TCN-short (5 lags), TCN-med (22 lags), and TCN-long (60 lags). By maintaining a consistent 22-lag input for the baseline models, we ensure a direct performance comparison with the TCN-med configuration. To ensure the robustness of our results while strictly adhering to chronological constraints to prevent data leakage, we implement a rolling-window scheme. For each estimation window ending at time t_2 , the data are partitioned as follows: JS:Maybe illustrate with a small tikz picture (just a time line with indicated dates)

- **Testing set:** The final six months of the window, $(t_2 - \Delta_{\text{test}}, t_2]$, reserved for an objective evaluation of out-of-sample performance;
- **Validation set:** The six-month period immediately preceding the testing set, $(t_2 - \Delta_{\text{test}} - \Delta_{\text{val}}, t_2 - \Delta_{\text{test}}]$, utilized for hyperparameter tuning of the neural networks (e.g., LSTM and TCN);
- **Training set:** The initial period from t_1 up to the start of the validation set, $[t_1, t_2 - \Delta_{\text{test}} - \Delta_{\text{val}}]$, used to estimate the parameters of each candidate model,

where Δ_{test} and Δ_{val} denote the lengths of the testing and validation windows, respectively. In our empirical setup, both are fixed at a duration of six months. Notably, for linear statistical models (OLS and HAR) that do not require hyperparameter tuning, the validation set is merged with the training set. This allows these models to

be estimated over the full sample period $[t_1, t_2 - \Delta_{\text{test}}]$, thereby maximizing the use of available historical information. In contrast, for deep learning models, the training and validation sets are kept separate to facilitate robust hyperparameter selection and mitigate overfitting.

To maintain adaptiveness in the presence of structural changes, we adopt a rolling retraining scheme with a stride of six months. After completing the evaluation over a given test window, the estimation window is advanced by six months, and all models are re-trained using the updated dataset. The initial window spans from January 1, 2016, to December 31, 2019. This procedure generates a sequence of pre-trained model configurations, which define the action space for the reinforcement learning agent. The agent then performs adaptive model selection over the out-of-sample period from January 1, 2020, to December 31, 2024.

4.2.2 Training and Evaluation of the DQN Agent

Upon establishing the forecasting library, we deploy the RL framework for dynamic model selection with a look-back window of $D = 60$. At each step t , the agent observes the state s_t —integrating historical trajectories and prospective insights—and executes an action a_t to select from $K = 6$ candidate models. The resulting transition and policy optimization are governed by the reward signal defined in (2). [Specifically, Patton and Sheppard \(2009\) shows that the QLIKE loss function possesses desirable robustness properties for volatility forecast evaluation. Therefore, in our empirical implementation, we instantiate the loss function \(1\) in the reward construction using the QLIKE criterion:](#)

$$e_t^{(j)} = \frac{\exp(V_{t+1})}{\exp(\hat{V}_{t+1}^{(j)})} - (V_{t+1} - \hat{V}_{t+1}^{(j)}) - 1, \quad j = 1, \dots, K.$$

To ensure a sufficient sample size for the deep neural network and to enhance the agent’s generalization capability across different market conditions, we pool the observations from all stocks into a consolidated cross-sectional dataset. We partition this dataset into three disjoint subsets—training, validation, and testing—maintaining a strict chronological order:

- **Testing set:** The final six months ending on December 31, 2024, used for the final out-of-sample evaluation.
- **Validation set:** The six months immediately preceding the testing period, utilized for hyperparameter tuning and early stopping of the DQN.
- **Training set:** All observations from January 1, 2020, up to the start of the validation set, dedicated to the optimization of the Q-network.

During the training phase, we adopt a randomized experience replay strategy. In each iteration, a transition (s, a, r, s') is sampled from the pool of stocks and time steps within the training set. For multi-step tasks ($h \geq 1$), an episode is initialized at a random time t and proceeds over a h -step horizon to account for the recursive feedback mechanism. This chronological partitioning, combined with the randomized sampling from the consolidated pool, ensures that the agent learns a robust, universal selection logic while strictly preventing data leakage.

4.3 Evaluation

To benchmark the performance of the RLVF framework, we compare it against two classes of baselines:

- **Individual Base Models:** The six pre-trained models: OLS, HAR, LSTM, TCN-short, TCN-med and TCN-long. These serve to demonstrate whether the RL agent can consistently outperform the best single model in the pool.
- **Model Averaging (Ensemble):** A simple equally weighted average (EW) of all candidate forecasts. This baseline represents a static combination strategy and helps verify the value of the agent’s dynamic, state-dependent selection logic. [ZS:](#)
- **Supervised Model Selection:** An XGBoost classifier that uses the same state information as the RLVF agent to select one model from the candidate pool at each decision time. This benchmark represents a flexible state-dependent supervised selection rule and allows us to assess whether the sequential learning structure of RLVF provides predictive gains beyond conventional supervised model selection.

ZS:Consistent with the volatility forecasting literature (Patton, 2011), we evaluate predictive accuracy using the QLIKE loss function and the Mean Squared Error (MSE). To assess whether differences in forecasting performance are statistically significant, we further conduct the modified Diebold-Mariano (DM) test (Gu et al., 2020) based on the forecast losses computed over the test sample. Following Patton and Sheppard (2009), we place greater emphasis on the results based on the QLIKE loss, as it has been shown to provide a more powerful and robust basis for comparing volatility forecasts.

4.3.1 Case I: Direct Forecasting ($h = 1$)

We first examine the direct forecasting scenario ($h = 1$), where the RL agent identifies the optimal model for the immediate next trading period (i.e., the next hour or day). The MSE and QLIKE results are reported in Table 1, and the corresponding modified DM test results are reported in Table 2. **ZS:From Table 2, the RL agent exhibits substantially stronger performance under the QLIKE loss. For the 1-day horizon, it significantly outperforms all benchmark models. For the 1-hour horizon, significant improvements are observed relative to EW, OLS, HAR, and TCN-Short, while no significant differences are detected with respect to LSTM, TCN-Med, and TCN-Long. Overall, the results suggest that the RL agent is particularly effective in improving volatility forecasts when evaluated using the QLIKE loss, which is widely regarded as a more appropriate loss function for volatility forecasting.**JS:We should also discuss the performance under MSE

Table 1: **One-step-ahead forecasting performance measured by MSE and QLIKE.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
RLVF	0.3741	0.6427	0.2246	0.3617
XGBoost	0.3784	0.7018	0.2357	0.4034
EW	0.3713	0.6157	0.2338	0.3881
OLS	0.3713	0.6443	0.2324	0.4141
HAR	0.3729	1.1416	0.2337	0.8508
LSTM	0.3677	0.5877	0.2333	0.3664
TCN-Short	0.3790	0.6975	0.2326	0.4515
TCN-Med	0.3735	0.5861	0.2369	0.3619
TCN-Long	0.3756	0.5802	0.2413	0.3735

Table 2: **Modified Diebold–Mariano test for one-step-ahead forecasting accuracy.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
XGBoost	-1.5555*	-6.9472***	-3.6934***	-5.3352***
EW	0.6833	3.3717	-2.8099***	-3.3021***
OLS	0.6557	0.3720	-2.1748**	-4.5480***
HAR	0.1681	-13.8528***	-1.9265**	-10.4356***
LSTM	1.2589	6.7717	-2.4471***	-0.8471
TCN-Short	-1.9355**	-3.1907***	-4.3050***	-7.2323***
TCN-Med	0.1189	7.3016	-3.4812***	0.3224
TCN-Long	-0.2834	6.3993	-3.3510***	-1.5848*

Notes: The table reports modified Diebold–Mariano (DM) test statistics for pairwise comparisons between the RL agent and each benchmark model using MSE and QLIKE losses. The loss differential is defined as the loss of the RL agent minus the loss of the benchmark model. The tests are conducted as one-sided tests against the alternative hypothesis that the RL agent has a lower expected predictive loss than the corresponding benchmark. Statistical significance is denoted by stars, where ***, **, and * correspond to rejection of the null hypothesis at the 1%, 5%, and 10% levels, respectively.

JS:I think we should report the action distribution? Currently commented out. It is important to show that the agent is not just picking one model all the time.

4.3.2 Case II: Recursive Forecasting ($h = 5$)

We next examine the recursive forecasting setting with a five-step forecasting episode ($h = 5$). Since the RL agent is trained over five recursive forecasting steps, we first evaluate its performance using losses aggregated over the entire episode, which is consistent with the training objective. To further examine how predictive accuracy evolves within the episode, we also report step-specific average losses and modified Diebold–Mariano tests for each recursive step.

The aggregate loss and the corresponding DM test results are reported in Tables 3 and 4, respectively. The step-wise average losses and their corresponding DM test results are presented in Tables 5 and 6, respectively. ZS:Several findings emerge from the recursive forecasting results. First, the conclusions are broadly consistent with those obtained under direct forecasting. Although the RL agent does not achieve the lowest MSE among all competing methods, it consistently delivers the best performance under the QLIKE loss for both the 1-day and 1-hour targets. Second, the step-wise DM tests reveal that the relative performance of the RL agent evolves differently across loss functions. Under MSE, the agent is not uniformly superior and is often outperformed by several benchmark models as the recursive forecasting horizon increases. In contrast, under QLIKE, the RL agent exhibits remarkably stable and statistically significant improvements over most benchmark models across all recursive forecasting stages. These findings suggest that the RL-based model selection strategy is particularly effective at improving volatility forecasts when evaluated using QLIKE, which is generally regarded as a more appropriate loss function for volatility forecasting.

JS:How are the results in Table 3 computed? The numbers do not appear to be averages of the next table. For example, the five RLVF 1-day MSE entries average to approximately 0.4674 0.4674, whereas the aggregate table reports 0.4736 0.4736; the 1-hour QLIKE entries average to approximately 0.3936 0.3936, versus 0.3806 0.3806 in the aggregate table. Define the aggregation formula, discounting, weights, and sample counts, or regenerate the table from the same evaluation output. ZS:Thank you for pointing this out. The discrepancy was caused by the use of two different aggregation schemes. The previous aggregate table was computed directly from the evaluation

script by pooling all available stock–date–horizon observations:

$$\hat{L}_{\text{old}} = \frac{1}{N} \sum_{h=1}^K \sum_{d \in \mathcal{D}_h} \sum_{s \in \mathcal{S}_{d,h}} L_{s,d,h}, \quad N = \sum_{h=1}^K \sum_{d \in \mathcal{D}_h} N_{s,d,h},$$

where $K = 5$, $\mathcal{D}_h$ is the set of evaluation dates available at horizon h , $\mathcal{S}_{d,h}$ is the set of evaluation stocks available at horizon h and date d , and $N_{s,d,h}$ is the number of available stocks on date d at horizon h . Under this definition, each individual stock–date–horizon observation receives the same weight $1/N$. We have now revised the aggregate table using the following aggregation scheme:

$$\hat{L}_{\text{new}} = \frac{1}{K} \sum_{h=1}^K \frac{1}{N_{d,h}} \sum_{d \in \mathcal{D}_h} \frac{1}{N_{s,d,h}} \sum_{s \in \mathcal{S}_{d,h}} L_{s,d,h}.$$

The two definitions generally differ when the numbers of available stocks or evaluation dates vary across dates or horizons, because the old definition assigns equal weight to each individual prediction, whereas the revised definition assigns equal weight to each horizon and each evaluation date. We have updated the aggregate results accordingly, so that they are now exactly equal to the arithmetic averages of the five corresponding horizon-specific losses.

Table 3: **Five-step-ahead recursive forecasting performance measured by MSE and QLIKE.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
RLVF	0.4674	0.7776	0.3112	0.3936
XGBoost	0.4716	0.7981	0.3267	0.4767
EW	0.4599	0.6946	0.3222	0.4619
OLS	0.4604	0.7189	0.3188	0.4780
HAR	0.4606	1.1695	0.3190	0.8732
LSTM	0.4534	0.6757	0.3221	0.4547
TCN-Short	0.4780	0.7978	0.3187	0.5418
TCN-Med	0.4626	0.6718	0.3279	0.4458
TCN-Long	0.4699	0.6649	0.3432	0.4592

Notes: This table reports the aggregate MSE and QLIKE losses of each model over the entire recursive forecasting path, comprising stages $h' = 0, \dots, 4$, under the 1-day and 1-hour forecasting

settings. The aggregate loss is obtained by averaging the stage-specific losses over the full forecasting path, with smaller values indicating better overall predictive accuracy. The corresponding modified Diebold–Mariano tests reported in Table 4 evaluate whether the differences in aggregate forecasting performance are statistically significant.

Table 4: **Modified Diebold–Mariano test for Five-step-ahead recursive forecasting accuracy.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
XGBoost	-0.969	-1.659**	-3.412***	-4.112***
EW	1.602	5.784	-2.626***	-4.365***
OLS	1.621	3.650	-1.809**	-3.857***
HAR	1.572	-19.419***	-1.804**	-18.196***
LSTM	2.087	5.681	-2.056**	-2.908***
TCN-Short	-1.563*	-1.156	-0.993	-5.767***
TCN-Med	0.872	6.579	-2.942***	-3.003***
TCN-Long	-0.251	5.787	-2.421***	-2.587***

Notes: The table reports modified Diebold–Mariano (DM) test statistics for pairwise comparisons between the RL agent and each benchmark model using MSE and QLIKE losses. The loss differential is defined as the loss of the RL agent minus the loss of the benchmark model. The tests are conducted as one-sided tests against the alternative hypothesis that the RL agent has a lower expected predictive loss than the corresponding benchmark. Statistical significance is denoted by stars, where ***, **, and * correspond to rejection of the null hypothesis at the 1%, 5%, and 10% levels, respectively.

Table 5: **Step-wise average loss.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
$h' = 0$				
RLVF	0.3778	0.7332	0.2298	0.3640
XGBoost	0.3795	0.7520	0.2377	0.4283
EW	0.3741	0.6490	0.2363	0.4015
OLS	0.3742	0.6781	0.2350	0.4290

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
HAR	0.3761	1.1798	0.2364	0.8608
LSTM	0.3706	0.6215	0.2359	0.3820
TCN-Short	0.3816	0.7278	0.2345	0.4637
TCN-Med	0.3762	0.6198	0.2392	0.3764
TCN-Long	0.3787	0.6133	0.2442	0.3903
$h' = 1$				
RLVF	0.4477	0.7683	0.2882	0.3878
XGBoost	0.4539	0.7897	0.3045	0.4622
EW	0.4411	0.6980	0.2984	0.4451
OLS	0.4414	0.7176	0.2959	0.4645
HAR	0.4442	1.1823	0.3006	0.8621
LSTM	0.4350	0.6772	0.2975	0.4352
TCN-Short	0.4541	0.7993	0.2940	0.5209
TCN-Med	0.4436	0.6743	0.3024	0.4260
TCN-Long	0.4487	0.6636	0.3131	0.4349
$h' = 2$				
RLVF	0.4892	0.7860	0.3294	0.3989
XGBoost	0.4935	0.8077	0.3439	0.4834
EW	0.4811	0.6980	0.3409	0.4676
OLS	0.4816	0.7194	0.3375	0.4842
HAR	0.4810	1.1238	0.3372	0.8224
LSTM	0.4741	0.6811	0.3410	0.4598
TCN-Short	0.5013	0.8140	0.3367	0.5618
TCN-Med	0.4839	0.6761	0.3474	0.4492
TCN-Long	0.4908	0.6720	0.3624	0.4674
$h' = 3$				
RLVF	0.5043	0.8061	0.3496	0.4081
XGBoost	0.5092	0.8182	0.3676	0.5013
EW	0.4945	0.6980	0.3614	0.4832

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
OLS	0.4951	0.7195	0.3567	0.4942
HAR	0.4936	1.1020	0.3541	0.8219
LSTM	0.4870	0.6873	0.3620	0.4847
TCN-Short	0.5186	0.8082	0.3583	0.5589
TCN-Med	0.4976	0.6823	0.3688	0.4748
TCN-Long	0.5067	0.6799	0.3884	0.4965
$h' = 4$				
RLVF	0.5180	0.7942	0.3591	0.4093
XGBoost	0.5219	0.8232	0.3798	0.5085
EW	0.5086	0.7296	0.3742	0.5123
OLS	0.5099	0.7600	0.3689	0.5180
HAR	0.5083	1.2596	0.3665	0.9986
LSTM	0.5002	0.7113	0.3744	0.5117
TCN-Short	0.5343	0.8395	0.3699	0.6038
TCN-Med	0.5119	0.7065	0.3815	0.5024
TCN-Long	0.5246	0.6957	0.4080	0.5068

Notes: This table reports the average MSE and QLIKE losses of each model at each recursive forecasting stage $h' = 0, \dots, 4$ under the 1-day and 1-hour forecasting settings. Smaller values indicate better predictive accuracy. These step-wise losses provide the magnitude of forecasting performance differences, while the corresponding modified Diebold–Mariano tests in Table 6 evaluate their statistical significance.

Table 6: **Step-wise modified Diebold–Mariano test for recursive forecasting accuracy.**

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
$h' = 0$				
XGBoost	-0.525	-1.345*	-2.266**	-3.768***

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
EW	1.073	5.812	-2.523***	-3.318***
OLS	0.973	3.408	-1.808**	-4.505***
HAR	0.366	-12.800***	-1.523*	-11.198***
LSTM	1.638	6.663	-2.124**	-1.119
TCN-Short	-1.284*	0.315	-2.012**	-6.338***
TCN-Med	0.426	6.848	-3.318***	-0.813
TCN-Long	-0.179	6.412	-3.421***	-1.382*
$h' = 1$				
XGBoost	-1.212	-1.716**	-2.628***	-5.212***
EW	1.180	5.514	-2.547***	-6.017***
OLS	1.074	3.430	-1.793**	-4.781***
HAR	0.428	-13.505***	-1.494*	-12.050***
LSTM	1.705	5.412	-2.007***	-2.910***
TCN-Short	-1.080	-1.777**	-0.997	-8.036***
TCN-Med	0.720	6.253	-3.286***	-2.940***
TCN-Long	-0.115	6.089	-2.859***	-2.912***
$h' = 2$				
XGBoost	-0.994	-1.804**	-4.043***	-5.331***
EW	1.409	5.705	-2.543***	-4.860***
OLS	1.290	3.958	-1.651**	-4.071***
HAR	1.333	-13.775***	-1.489*	-13.302***
LSTM	1.821	5.653	-1.931**	-3.357***
TCN-Short	-1.439*	-1.381*	-0.824	-4.488***
TCN-Med	0.790	6.407	-2.750***	-3.061***
TCN-Long	-0.146	5.650	-2.387***	-3.198***
$h' = 3$				
XGBoost	-0.892	-0.817	-3.223***	-3.525***
EW	1.709	5.864	-2.417***	-3.832***
OLS	1.745	4.514	-1.569*	-3.678***

Model	MSE		QLIKE	
	1-day	1-hour	1-day	1-hour
HAR	2.204	-14.147***	-2.022**	-15.360***
LSTM	2.114	5.362	-2.025**	-3.174***
TCN-Short	-1.504*	-0.096	-0.898	-3.574***
TCN-Med	0.960	6.055	-2.538***	-3.170***
TCN-Long	-0.187	5.236	-2.220**	-2.832***
$h' = 4$				
XGBoost	-0.617	-2.081**	-2.800***	-4.543***
EW	1.443	3.214	-2.301**	-4.534***
OLS	1.613	1.748	-1.719**	-4.008***
HAR	1.987	-16.090***	-2.064**	-13.697***
LSTM	1.965	3.344	-2.041**	-3.216***
TCN-Short	-1.546*	-1.965**	-0.995	-6.925***
TCN-Med	0.776	3.853	-2.443***	-3.285***
TCN-Long	-0.418	3.884	-2.179**	-2.836***

Notes: The table reports modified Diebold–Mariano (DM) test statistics for pairwise comparisons between the RLVF and each benchmark model. Results are reported separately for each recursive forecasting stage ($h' = 0, \dots, 4$) within a five-step forecasting episode. The tests are conducted as one-sided tests with the alternative hypothesis that the RL agent has a lower expected predictive loss than the corresponding benchmark model. Statistical significance is denoted by stars, where ***, **, and * correspond to rejection of the null hypothesis at the 1%, 5%, and 10% significance levels, respectively.

5 Conclusion

This paper develops a reinforcement-learning approach to realized volatility forecasting by casting model choice as a sequential decision problem. A heterogeneous library of six candidate forecasters is combined with a DQN agent that selects models based on recent volatility dynamics, model-specific error histories, and contemporaneous candidate forecasts. The resulting framework accommodates both one-step and recursive

multi-step prediction while preserving a clear separation between base-model estimation and policy learning.

The empirical study on 40 Chinese stocks related to the SSE 50 Index indicates that dynamic model selection is a viable alternative to fixed single-model forecasts and static equal-weight combinations. In the reported experiments, the RL agent delivers particularly strong QLIKE performance, while the strongest benchmark can still remain competitive under MSE. These results suggest that reinforcement learning is most useful here as an adaptive selection layer that exploits model complementarity across market states. Future work can extend the analysis to additional recursive horizons, broader asset universes, and alternative reward specifications.

To Do

- Resolve the recursive reward-objective issue. Determine whether subtracting the ex-post minimum candidate loss at every recursively generated state preserves the desired policy ordering; otherwise use the negative forecast loss as the recursive reward or restate the estimand as cumulative selection regret and revise the theoretical claims accordingly.
- Use consistent terminology for realized variance, realized volatility, and log realized variance throughout.
- Document the complete rolling chronology for base-model estimation, forecast generation, DQN training, validation, and final testing. Relabel intermediate “test” windows as out-of-fold forecast-generation windows where appropriate and add a timeline showing that no future outcomes enter any state, forecast, or tuning decision.
- Give the exact OLS and HAR equations; LSTM, TCN, DQN, and XGBoost specifications in an appendix.
- Repeat neural-network experiments across multiple random seeds and report means and uncertainty.
- Justify the current pooled modified Diebold–Mariano inference.

- Demonstrate that the learned policy is genuinely adaptive. Report overall and regime-conditional action distributions, switching frequencies, conditional losses, and the attainable ex-post oracle gap rather than only aggregate forecast losses.

Acknowledgements

J. Sester gratefully acknowledges support by the NUS Start-Up Grant *Tackling model uncertainty in Finance with machine learning*, by the MOE AcRF Tier 1 Grant 25-0428-P0001, and by the MOE AcRF Tier 2 Grant T2EP20225-0030.

References

- Andersen, T. G., T. Bollerslev, F. X. Diebold, and H. Ebens (2001). The distribution of realized stock return volatility. *Journal of Financial Economics* 61(1), 43–76.
- Andersen, T. G., T. Bollerslev, F. X. Diebold, and P. Labys (2003). Modeling and forecasting realized volatility. *Econometrica* 71(2), 579–625.
- Bollerslev, T. (1986a). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* 31(3), 307–327.
- Bollerslev, T. (1986b). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* 31(3), 307–327.
- Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics* 18(3), 502–531.
- Chien, C.-F., Y.-S. Lin, and S.-K. Lin (2020). Deep reinforcement learning for selecting demand forecast models to empower industry 3.5 and an empirical study for a semiconductor component distributor. *International Journal of Production Research* 58(9), 2784–2804.
- Christensen, K., M. Siggaard, and B. Veliyev (2023). A machine learning approach to volatility forecasting. *Journal of Financial Econometrics* 21(5), 1680–1727.
- Christoffersen, P., K. Jacobs, and K. Mimouni (2010). Volatility dynamics for the s&p500: Evidence from realized volatility, daily returns, and option prices. *The Review of Financial Studies* 23(8), 3141–3189.

- Corsi, F. (2009a). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics* 7(2), 174–196.
- Corsi, F. (2009b). A simple approximate long-memory model of realized volatility. *The Journal of Financial Econometrics* 7(2), 174–196.
- Feng, C., M. Sun, and J. Zhang (2020). Reinforced deterministic and probabilistic load forecasting via q-learning dynamic model selection. *IEEE Transactions on Smart Grid* 11(2), 1377–1386.
- Fu, Y., D. Wu, and B. Boulet (2022). Reinforcement learning based dynamic model combination for time series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 36(6), 6639–6647.
- Gu, S., B. Kelly, and D. Xiu (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies* 33(5), 2223–2273.
- Hansen, P. R., Z. Huang, and H. H. Shek (2012). Realized garch: a joint model for returns and realized measures of volatility. *Journal of Applied Econometrics* 27(6), 877–906.
- Hansen, P. R. and A. Lunde (2005). A forecast comparison of volatility models: Does anything beat a garch(1,1)? *Journal of Applied Econometrics* 20(7), 873–889.
- Kerschke, P., H. H. Hoos, F. Neumann, and H. Trautmann (2019). Automated algorithm selection: Survey and perspectives. *Evolutionary Computation* 27(1), 3–45.
- Koop, G. and D. Korobilis (2012). Forecasting inflation using dynamic model averaging. *International Economic Review* 53(3), 867–886.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovsky, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015). Human-level control through deep reinforcement learning. *Nature* 518(7540), 529–533.
- Pannakkong, W., V. T. Vinh, N. N. M. Tuyen, and J. Buddhakulsomsiri (2023). A reinforcement learning approach for ensemble machine learning models in peak electricity forecasting. *Energies* 16(13), 5099.

- Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of econometrics* 160(1), 246–256.
- Patton, A. J. and K. Sheppard (2009). Evaluating volatility and correlation forecasts. In *Handbook of financial time series*, pp. 801–838. Springer.
- Poon, S.-H. and C. W. J. Granger (2003). Forecasting volatility in financial markets: A review. *Journal of Economic Literature* 41(2), 478–539.
- Raftery, A. E., M. Kárný, and P. Ettler (2010). Online prediction under model uncertainty via dynamic model averaging: Application to a cold rolling mill. *Technometrics* 52(1), 52–66.
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction* (2 ed.). MIT Press.
- Zhang, C., Y. Zhang, M. Cucuringu, and Z. Qian (2024). Volatility forecasting with machine learning and intraday commonality. *Journal of Financial Econometrics* 22(2), 492–530.